

Week 10 Skill

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

```
divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ pwd
/home/divya/job_listing
divya@Divyasree:~/job_listing$ nano signal_handlers.c
divya@Divyasree:~/job_listing$ gcc signal_handlers.c -o signal_handlers
divya@Divyasree:~/job_listing$ ./signal_handlers
===== SIGNAL HANDLER MANAGEMENT =====
Process PID: 561

[1] Four signal handlers registered.

Signals configured:
SIGINT -> Ctrl+C
SIGTERM -> Termination signal
SIGUSR1 -> User-defined signal 1
SIGUSR2 -> User-defined signal 2

===== SIGNAL TEST MODE =====
Process is stable and waiting for signals.
Use another terminal to send SIGUSR1/SIGUSR2/SIGTERM.
You can press Ctrl+C to test SIGINT.

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...
^C
[HANDLER] Signal received!
Signal Number : 2
Signal Count : 1
Sender PID : 0
Signal Type : SIGINT (Interrupt)
Signal handled successfully.

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...

Waiting for signal...
```

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

```
divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ nano interrupt_job.c
divya@Divyasree:~/job_listing$ gcc interrupt_job.c -o interrupt_job
divya@Divyasree:~/job_listing$ ./interrupt_job
===== SIGINT FOREGROUND JOB CONTROL =====
[Shell] Shell PID: 540

[Shell] Foreground Job Created.
[Shell] Job PID: 541
[Shell] Job State: Running
[Shell] Terminal control transferred to job.

[Job] Foreground job started.
[Job] PID: 541
[Job] Process Group ID: 541
[Job] Running...
[Job] Press Ctrl+C to terminate this foreground job.
[Job] Working... 1/20
[Job] Working... 2/20
[Job] Working... 3/20
[Job] Working... 4/20
[Job] Working... 5/20
[Job] Working... 6/20
[Job] Working... 7/20
[Job] Working... 8/20
[Job] Working... 9/20
[Job] Working... 10/20
[Job] Working... 11/20
[Job] Working... 12/20
[Job] Working... 13/20
[Job] Working... 14/20
[Job] Working... 15/20
[Job] Working... 16/20
[Job] Working... 17/20
[Job] Working... 18/20
[Job] Working... 19/20
[Job] Working... 20/20
[Job] Completed normally.

[Shell] Terminal control returned to shell.
[Shell] Foreground job exited normally.
[Job Table] Job State: Completed

===== FINAL JOB STATUS =====
PID : 541
State : Completed

[Shell] Interrupt handling test completed.
divya@Divyasree:~/job_listing$ |
```

Code :

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t foreground_pid = -1;
int job_state = 0;

/* SIGINT handler for the shell */
void handle_sigint(int sig)
{
    printf("\n[Shell] SIGINT received.\n");

    if (foreground_pid > 0) {
        printf("[Shell] Forwarding SIGINT to foreground job PID %d...\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
}

int main()
{
    pid_t pid;
    int status;

    printf("==== SIGINT FOREGROUND JOB CONTROL =====\n");

    /* Therefore Ctrl+C will terminate this job. */
    signal(SIGINT, SIG_DFL);

    printf("\n[Job] Foreground job started.\n");
    printf("[Job] PID: %d\n", getpid());
    printf("[Job] Process Group ID: %d\n", getpgid());

    printf("[Job] Running...\n");
    printf("[Job] Press Ctrl+C to terminate this foreground job.\n");

    for (int i = 1; i <= 20; i++) {
        printf("[Job] Working... %d/20\n", i);
        sleep(1);
    }

    printf("[Job] Completed normally.\n");

    exit(0);
}

/* ===== PARENT / SHELL ===== */

foreground_pid = pid;

/*
 * Put child in its own process group.
 */
setpgid(pid, pid);

/* Give terminal control to the foreground job. */
tcsetpgid(STDIN_FILENO, pid);

printf("[Shell] Terminal control transferred to job.\n");

/*
 * Wait for the foreground job.
 */
waitpid(pid, &status, WUNTRACED);

/*
 * Take terminal control back.
 */
tcsetpgid(STDIN_FILENO, getpgid());

printf("\n[Shell] Terminal control returned to shell.\n");

/*
 * Check how the job ended.
 */
if (WIFSIGNALED(status)) {
    printf("[Shell] Foreground job terminated by signal %d.\n",
           WTERMSIG(status));

    job_state = 3;

    printf("[Job Table] Job State: Terminated\n");
}

```

```

    }
    else if (WIFSTOPPED(status)) {

        printf("[Shell] Foreground job stopped.\n");

        job_state = 2;

        printf("[Job Table] Job State: Stopped\n");
    }
    else if (WIFEXITED(status)) {

        printf("[Shell] Foreground job exited normally.\n");

        job_state = 4;

        printf("[Job Table] Job State: Completed\n");
    }

    foreground_pid = -1;

    printf("\n==== FINAL JOB STATUS =====\n");

    printf("PID    : %d\n", pid);

    if (job_state == 3)
        printf("State : Terminated by SIGINT\n");
    else if (job_state == 2)
        printf("State : Stopped\n");
    else if (job_state == 4)
        printf("State : Completed\n");

    printf("\n[Shell] Interrupt handling test completed.\n");

    return 0;
}

```

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```

divya@Divyasree:~$ ^C
divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ nano suspend_job.c
divya@Divyasree:~/job_listing$ gcc suspend_job.c -o suspend_job
divya@Divyasree:~/job_listing$ ./suspend_job
===== SIGTSTP JOB SUSPENSION =====
[Shell] Shell PID: 809

[Job Table]
Job ID : 1
PID    : 810
State  : Running

[Shell] Job is running.
[Shell] Press Ctrl+Z to suspend the job.

[Job] Process started.
[Job] PID: 810
[Job] Process Group: 810
[Job] Working... Step 1/20
[Job] Working... Step 2/20
[Job] Working... Step 3/20
[Job] Working... Step 4/20
[Job] Working... Step 5/20
[Job] Working... Step 6/20
[Job] Working... Step 7/20
[Job] Working... Step 8/20
[Job] Working... Step 9/20
[Job] Working... Step 10/20
[Job] Working... Step 11/20
^Z

[Shell] SIGTSTP received (Ctrl+Z).
[Shell] SIGTSTP received (Ctrl+Z).
[Shell] Forwarding SIGTSTP to Job PID 810...

[Job Table] Job 1 -> Stopped
[Shell] Job has been suspended.

===== STATE PRESERVED =====
[Shell] PID 810 is stopped.
[Shell] Process state is preserved in memory.

Resume this job now?
1. Resume Job
2. Exit
Enter choice: 1

[Shell] Sending SIGCONT to Job 810...
[Job Table] Job 1 -> Running
[Shell] Job resumed successfully.
[Job] Working... Step 12/20
[Job] Working... Step 13/20
[Job] Working... Step 14/20
[Job] Working... Step 15/20
[Job] Working... Step 16/20
[Job] Working... Step 17/20
[Job] Working... Step 18/20
[Job] Working... Step 19/20
[Job] Working... Step 20/20

[Job] Job completed normally.

[Job Table] Job 1 -> Completed
[Shell] Job completed successfully.

```

```
[Job Table] Job 1 -> Completed
[Shell] Job completed successfully.

===== FINAL JOB STATUS =====
Job ID : 1
PID    : 810
State  : Completed

[Shell] Suspension workflow test completed.
divya@Divyasree:~/job_listing$ |
```

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/* Job states */
char job_state[20] = "Running";

/* Capture Ctrl+Z */
void handle_sigstsp(int sig)
{
    printf("\n\n[Shell] SIGTSTP received (Ctrl+Z).\n");

    if (foreground_pid > 0) {
        printf("[Shell] Forwarding SIGTSTP to Job PID %d...\n",
            foreground_pid);

        kill(foreground_pid, SIGTSTP);
    }
}

int main()
{
    pid_t pid;
    int status;
    int choice;
```

```
/* Create child process */
pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

/* ===== CHILD ===== */

if (pid == 0) {

    /* Create separate process group */
    setpgid(0, 0);

    /* Child must use default SIGTSTP behavior */
    signal(SIGTSTP, SIG_DFL);

    printf("\n[Job] Process started.\n");
    printf("[Job] PID: %d\n", getpid());
    printf("[Job] Process Group: %d\n", getpgrp());

    for (int i = 1; i <= 20; i++) {
        printf("[Job] Working... Step %d/20\n", i);
        sleep(1);
    }

    printf("\n[Job] Job completed normally.\n");
    exit(0);
}

/* ===== PARENT / SHELL ===== */

foreground_pid = pid;

/* Put child in separate process group */
setpgid(pid, pid);

printf("\n[Job Table]\n");
printf("Job ID : 1\n");
printf("PID    : %d\n", foreground_pid);
printf("State   : %s\n", job_state);

printf("\n[Shell] Job is running.\n");
printf("[Shell] Press Ctrl+Z to suspend the job.\n");

/*
 * Wait until the child either stops or exits.
 */
waitpid(pid, &status, WUNTRACED);
```

```

printf("\n[Job Table] Job 1 -> %s\n", job_state);

printf("[Shell] Job has been suspended.\n");

printf("\n==== STATE PRESERVED =====\n");
printf("[Shell] PID %d is stopped.\n", foreground_pid);
printf("[Shell] Process state is preserved in memory.\n");

printf("\nResume this job now?\n");
printf("1. Resume Job\n");
printf("2. Exit\n");
printf("Enter choice: ");

scanf("%d", &choice);

if (choice == 1) {

    printf("\n[Shell] Sending SIGCONT to Job %d...\n",
        foreground_pid);

    kill(foreground_pid, SIGCONT);

    /* Wait for continued status */
    waitpid(pid, &status, WCONTINUED);

    if (WIFCONTINUED(status)) {

        snprintf(job_state,
            sizeof(job_state),
            "Running");

        printf("[Job Table] Job 1 -> %s\n",
            job_state);

        printf("[Shell] Job resumed successfully.\n");

    }

    /* Wait for final completion */
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {

        snprintf(job_state,
            sizeof(job_state),
            "Completed");

        printf("\n[Job Table] Job 1 -> %s\n",
            job_state);

        printf("[Shell] Job completed successfully.\n");

    }

    printf("\n[Shell] Job remains stopped.\n");

}

else if (WIFEXITED(status)) {

    snprintf(job_state,
        sizeof(job_state),
        "Completed");

    printf("\n[Job Table] Job 1 -> %s\n",
        job_state);

}

printf("\n==== FINAL JOB STATUS =====\n");
printf("Job ID : 1\n");
printf("PID : %d\n", foreground_pid);
printf("State : %s\n", job_state);

printf("\n[Shell] Suspension workflow test completed.\n");

return 0;
}

```

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior


```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t job_pgid = -1;

/* Signal handler for the shell */
void shell_sigint(int sig)
{
    printf("\n[Shell] SIGINT received.\n");

    if (job_pgid > 0) {
        printf("[Shell] Sending SIGINT to process group %d...\n",
            job_pgid);
        kill(-job_pgid, SIGINT);
    }
}

/* Signal handler for child jobs */
void child_sigint(int sig)
{
    printf("\n[Job %d] SIGINT received.\n", getpid());
    printf("[Job %d] Terminating...\n", getpid());
    exit(0);
}

int main()
{
    pid_t child1, child2;
    int status;

    printf("==== PROCESS GROUP & TERMINAL CONTROL =====\n");
    printf("[Shell] Shell PID : %d\n", getpid());
    printf("[Shell] Shell PGID : %d\n", getpgid());
    printf("[Shell] Terminal PGID : %d\n", tcgetpgrp(STDIN_FILENO));

    /* Protect shell from SIGINT */
    signal(SIGINT, shell_sigint);

    /* Ignore SIGTTOU while changing terminal ownership */
    signal(SIGTTOU, SIG_IGN);

    /* ===== CHILD 1 ===== */
    child1 = fork();

    if (child1 < 0) {
        perror("fork");
        return 1;
    }

    if (child1 == 0) {
        /*
         * Child 1 creates a new process group.
         */
        setpgid(0, 0);

        signal(SIGINT, child_sigint);

        printf("\n[Job 1] PID : %d\n", getpid());
        printf("[Job 1] PGID : %d\n", getpgid());

        while (1) {
            printf("[Job 1] Running...\n");
            sleep(2);
        }
    }
}

```

```

GNU nano 7.2 process_groups.c
    printf("[Job 1] PGID : %d\n", getpgid());

    while (1) {
        printf("[Job 1] Running...\n");
        sleep(2);
    }
}

/*
 * Parent assigns child1's PID as the process group ID.
 */
job_pgid = child1;

setpgid(child1, job_pgid);

/* ===== CHILD 2 ===== */
child2 = fork();

if (child2 < 0) {
    perror("fork");
    return 1;
}

if (child2 == 0) {
    /*
     * Add child 2 to the same process group.
     */
    setpgid(0, job_pgid);

    signal(SIGINT, child_sigint);

    printf("\n[Job 2] PID : %d\n", getpid());
    printf("[Job 2] PGID : %d\n", getpgid());

    while (1) {
        printf("[Job 2] Running...\n");
        sleep(2);
    }
}

/*
 * Parent ensures child 2 belongs to the same group.
 */
setpgid(child2, job_pgid);

printf("\n==== PROCESS GROUP CREATED =====\n");

printf("[Shell] Job Process Group ID: %d\n", job_pgid);
printf("[Shell] Child 1 PID : %d\n", child1);
printf("[Shell] Child 2 PID : %d\n", child2);

/*
 * Transfer terminal control to job group.
 */
printf("\n[Shell] Transferring terminal control...\n");
tcsetpgrp(STDIN_FILENO, job_pgid);

printf("[Shell] Terminal PGID is now: %d\n",
    tcgetpgrp(STDIN_FILENO));

printf("\n[Shell] Both jobs are running in the same process group.\n");
printf("[Shell] Press Ctrl+C to send SIGINT to the entire group.\n");

/*
 * Wait for child 1.
 */
waitpid(child1, &status, 0);

```

```

/*
setpgid(child2, job_pgid);

printf("\n==== PROCESS GROUP CREATED =====\n");

printf("[Shell] Job Process Group ID: %d\n", job_pgid);
printf("[Shell] Child 1 PID      : %d\n", child1);
printf("[Shell] Child 2 PID      : %d\n", child2);

/*
 * Transfer terminal control to job group.
 */
printf("[Shell] Transferring terminal control...\n");
tcsetpgrp(STDIN_FILENO, job_pgid);

printf("[Shell] Terminal PGID is now: %d\n",
       tcgetpgrp(STDIN_FILENO));

printf("[Shell] Both jobs are running in the same process group.\n");
printf("[Shell] Press Ctrl+C to send SIGINT to the entire group.\n");

/*
 * Wait for child 1.
 */
waitpid(child1, &status, 0);

/*
 * Wait for child 2.
 */
waitpid(child2, &status, 0);

/*
 * Restore terminal ownership.
 */
printf("[Shell] Restoring terminal control to shell...\n");
tcsetpgrp(STDIN_FILENO, getpgrp());

printf("[Shell] Terminal PGID restored to: %d\n",
       tcgetpgrp(STDIN_FILENO));

printf("\n==== VALIDATION =====\n");

if (tcgetpgrp(STDIN_FILENO) == getpgrp()) {
    printf("[Shell] Terminal ownership restored successfully.\n");
} else {
    printf("[Shell] Terminal ownership restoration failed.\n");
}

printf("[Shell] Process group test completed.\n");

return 0;
}

```